

```
In [1]: extra_skills = [

    # Programming Languages
    'python',
    'java',
    'c',
    'c++',
    'c#',
    'javascript',
    'typescript',
    'r',
    'scala',
    'go',
    'rust',
    'php',
    'kotlin',
    'swift',

    # Databases
    'sql',
    'mysql',
    'postgresql',
    'mongodb',
    'sqlite',
    'oracle',
    'redis',

    # Data Science
    'data science',
    'data analysis',
    'data analytics',
    'data preprocessing',
    'data cleaning',
    'data visualization',
    'feature engineering',
    'feature selection',

    # Machine Learning
    'machine learning',
    'deep learning',
    'artificial intelligence',
    'generative ai',
    'agentic ai',
    'nlp',
    'computer vision',
    'reinforcement learning',
    'supervised learning',
    'unsupervised learning',

    # Python Libraries
```

```
'numpy',
'pandas',
'matplotlib',
'seaborn',
'plotly',
'scipy',
'statsmodels',

# ML Libraries
'scikit-learn',
'tensorflow',
'keras',
'pytorch',
'xgboost',
'lightgbm',
'catboost',

# Big Data
'hadoop',
'spark',
'pyspark',
'hive',
'kafka',

# BI Tools
'power bi',
'tableau',
'looker',
'excel',

# Deployment
'streamlit',
'flask',
'fastapi',
'django',
'gradio',

# Cloud
'aws',
'azure',
'gcp',
'docker',
'kubernetes',

# ML Concepts
'regression',
'linear regression',
'logistic regression',
'decision tree',
'random forest',
'xgboost',
'clustering',
'k means clustering',
```

```
'dbscan',
'classification',
'cross validation',
'gridsearchcv',
'hyperparameter tuning',

# Statistics
'statistics',
'probability',
'hypothesis testing',
'a/b testing',

# Version Control
'git',
'github',
'gitlab',

# EDA
'exploratory data analysis',
'eda',

# Misc
'api',
'rest api',
'web scraping',
'beautifulsoup',
'selenium'
]
```

```
In [2]: import pandas as pd
import pdfplumber
import ast
```

```
In [3]: df = pd.read_csv('../data/processed/jobs_with_skills.csv')
```

```
In [4]: df['skills'] = df['skills'].apply(ast.literal_eval)
```

```
In [5]: type(df['skills'].iloc[0])
```

```
Out[5]: list
```

```
In [6]: all_skills = set()

for skills in df['skills']:
    all_skills.update(skills)

print("Total Skills:", len(all_skills))
all_skills.update(extra_skills)
all_skills = set(skill.lower() for skill in all_skills)
```

Total Skills: 42

```
In [7]: def extract_resume_text(pdf_path):

    text = ""

    with pdfplumber.open(pdf_path) as pdf:

        for page in pdf.pages:

            page_text = page.extract_text()

            if page_text:
                text += page_text + "\n"

    return text
```

```
In [8]: resume_text = extract_resume_text(
    "Resumes/resume_krishna.pdf"
)
```

```
In [9]: import re

def extract_skills_from_resume(text, skill_list):

    text = text.lower()

    found_skills = []

    for skill in skill_list:

        pattern = r'\b' + re.escape(skill.lower()) + r'\b'

        if re.search(pattern, text):

            found_skills.append(skill)

    return found_skills
```

```
In [10]: user_skills = extract_skills_from_resume(  
        resume_text,  
        all_skills  
    )
```

```
In [11]: user_skills
```

```
Out[11]: ['artificial intelligence',  
        'machine learning',  
        'data analysis',  
        'pandas',  
        'ml',  
        'data science',  
        'matplotlib',  
        'data visualization',  
        'streamlit',  
        'scikit-learn',  
        'data preprocessing',  
        'supervised learning',  
        'python',  
        'numpy',  
        'hyperparameter tuning',  
        'agentic ai',  
        'gridsearchcv',  
        'regression',  
        'clustering',  
        'github',  
        'seaborn',  
        'exploratory data analysis',  
        'eda',  
        'sql',  
        'api',  
        'data cleaning']
```

```
In [12]: import json  
  
with open("../data/processed/role_skills.json", "r") as f:  
    role_skills = json.load(f)
```

```
In [13]: role_skills.keys()
```

```
Out[13]: dict_keys(['Machine Learning Engineer', 'AI Engineer', 'Data  
Engineer', 'Data Scientist', 'Data Analyst', 'Analytics Engineer',  
'Backend Developer', 'Mlops Engineer', 'Python Developer', 'Business  
Analyst', 'Bi Analyst', 'Computer Vision Engineer', 'Software  
Engineer', 'Nlp Engineer'])
```

```
In [14]: len(role_skills)
```

```
Out[14]: 14
```

```
In [15]: list(role_skills.keys())[:10]
```

```
Out[15]: ['Machine Learning Engineer',  
          'AI Engineer',  
          'Data Engineer',  
          'Data Scientist',  
          'Data Analyst',  
          'Analytics Engineer',  
          'Backend Developer',  
          'Mlops Engineer',  
          'Python Developer',  
          'Business Analyst']
```

```
In [16]: role_skills['Data Scientist']
```

```
Out[16]: ['ml',  
          'machine learning',  
          'python',  
          'data science',  
          'generative ai',  
          'nlp',  
          'scala',  
          'llm',  
          'sql',  
          'excel']
```

```
In [17]: def skill_gap(user_skills, role, role_skills):  
  
          required_skills = role_skills[role]  
  
          missing_skills = []  
  
          for skill in required_skills:  
  
              if skill not in user_skills:  
  
                  missing_skills.append(skill)  
  
          return missing_skills
```

```
In [18]: missing_skills = skill_gap(
        user_skills,
        'Data Scientist',
        role_skills
    )

missing_skills
```

Out[18]: ['generative ai', 'nlp', 'scala', 'llm', 'excel']

```
In [19]: def match_score(user_skills, role, role_skills):

        required_skills = role_skills[role]

        if len(required_skills) == 0:
            return 0

        matched = 0

        for skill in required_skills:

            if skill in user_skills:

                matched += 1

        return (matched / len(required_skills)) * 100
```

```
In [20]: score = match_score(
        user_skills,
        'Data Scientist',
        role_skills
    )

score
```

Out[20]: 50.0

```
In [21]: role_scores = {}

        for role in role_skills:

            score = match_score(
                user_skills,
                role,
                role_skills
            )

            role_scores[role] = score
```

```
In [22]: scores_df = pd.DataFrame(  
    role_scores.items(),  
    columns=['Role', 'Score']  
)
```

```
In [23]: scores_df = scores_df.sort_values(  
    by='Score',  
    ascending=False  
)
```

```
In [24]: scores_df.head(10)
```

Out[24]:

	Role	Score
3	Data Scientist	50.0
0	Machine Learning Engineer	40.0
5	Analytics Engineer	40.0
4	Data Analyst	40.0
13	Nlp Engineer	40.0
7	Mlops Engineer	40.0
11	Computer Vision Engineer	30.0
1	AI Engineer	30.0
8	Python Developer	30.0
6	Backend Developer	30.0


```
In [25]: scores_df['Skill_Count'] = scores_df['Role'].apply(
        lambda x: len(role_skills[x])
    )

scores_df.head(10)
```

Out[25]:

	Role	Score	Skill_Count
3	Data Scientist	50.0	10
0	Machine Learning Engineer	40.0	10
5	Analytics Engineer	40.0	10
4	Data Analyst	40.0	10
13	Nlp Engineer	40.0	10
7	Mlops Engineer	40.0	10
11	Computer Vision Engineer	30.0	10
1	AI Engineer	30.0	10
8	Python Developer	30.0	10
6	Backend Developer	30.0	10

```
In [26]: scores_df.head(10)
```

Out[26]:

	Role	Score	Skill_Count
3	Data Scientist	50.0	10
0	Machine Learning Engineer	40.0	10
5	Analytics Engineer	40.0	10
4	Data Analyst	40.0	10
13	Nlp Engineer	40.0	10
7	Mlops Engineer	40.0	10
11	Computer Vision Engineer	30.0	10
1	AI Engineer	30.0	10
8	Python Developer	30.0	10
6	Backend Developer	30.0	10

In []:

Exported with [runcell](#) — convert notebooks to HTML or PDF anytime at [runcell.dev](#).